
Membership Inference Attack in Code

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 The copyright issue in the training corpora of large language models (LLMs) de-
2 serves serious attention. Respecting copyright and intellectual property is essential,
3 not only as a legal obligation, but also as a means of fostering a supportive and
4 respectful environment for content creators. Particularly, we focus our attention on
5 copyright (or copyleft) protection of code used during the pre-training process of
6 LLMs.

1 Introduction

8 The fundamentals of large language models (LLMs) stems from their access to vast and diverse
9 datasets for pre-training. However, the copyright issue in the training corpora deserves serious
10 attention. In recent years, numerous legal disputes have arisen concerning the use of copyrighted
11 materials in training data. Respecting copyright and intellectual property is essential, not only as
12 a legal obligation enforced by governmentscite, but also as a means of fostering a supportive and
13 respectful environment for content creators. As the era of LLMs advances and AI-generated content
14 increasingly populates the digital landscape, it becomes more important to uphold and protect human
15 creativity. Safeguarding originality ensures that human-generated works continue to contribute unique
16 and diverse perspectives to our culture and knowledge.

17 The programming capabilities of LLMs have inspired a range of prospective applications, from
18 autonomous agents to automated coders. These advancements rely on feeding LLMs with sufficient
19 knowledge from high-volume open-source code. However, such code is typically governed by specific
20 usage obligations. *e.g.*, copyleft licenses like the GNU General Public License (GPL). The recent
21 decades have seen many GPL conflicts and litigations [32]. We anticipate that similar open source
22 litigation will soon extend to LLM vendors. Looking back, the GPL was introduced in 1989 as part
23 of the GNU Project [31]. Eighteen years later, we witnessed the first recognized lawsuit for a GPL
24 violation, when Monsoon Multimedia was sued by the Software Freedom Conservancy [14, 8]. This
25 case underscores the importance for LLM developers to fully understand and respect the copyright
26 content [33]. Unfortunately, conflicts surrounding the use of pretraining data continue to emergecite,
27 and few have reached resolution due to the protracted nature of litigation.

28 A few research works have attempts to answer the transparency and accountability issues in LLMs.
29 Zhou et al. [38] analyzed books, websites, and Wikipedia to assess potential copyright violations.
30 Recent benchmarks [3, 11] support evaluation of both literal copying (*e.g.*, verbatim reproduction)
31 and non-literal copying (*e.g.*, replicated plots or characters) from LLM outputs. There are also
32 proactive measures known as copyright traps [22, 16], which embeds carefully crafted copyrighted
33 content (*i.e.*, similar to watermark) to enable precise detection of copyright violations. However, this
34 approach inherently suffers from the limitation that such traps must be prepared in advance.

35 A more promising approach involves applying membership inference attacks (MIAs) to LLMs. MIAs
36 were initially used to determine whether a specific data point was part of the training dataset of a
37 machine learning model [23, 25, 26, 7, 36, 10, 13, 17, 19, 20, 28], thereby exposing potential privacy
38 risks. MIAs in LLMs share the same intuitions that models memorizes data (overfitting) [24, 2].

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

39 Tirumala et al. [27] find that models memorize nouns and numbers, and that larger models can
40 memorize a larger portion of the data before over-fitting.

41 Unlike machine learning MIAs that rely on shadow or reference models, MIAs in LLMs typically
42 exploit the model’s token-level output and probabilities. Carlini et al. [2] demonstrated that LLMs
43 can leak verbatim training sentences. Nevertheless, identifying non-literal memorization remains
44 more challenging compared to literal (verbatim) memorization. **Most approaches focus on modeling**
45 **the likelihood of word sequences in context.** Duan [5] found that MIAs perform only slightly
46 better than random guessing in LLMs, due to the inherently fuzzy boundary between members and
47 non-members. Several approaches have been proposed, including leveraging neighboring data via
48 synonym substitution [15], calibrating token probabilities using term frequency divergence [37], and
49 detecting low-probability tokens [21]. However, they focus on general texts instead of code.

50 Yang et al. [34] presented GOTCHA, a membership inference attack tailored for code models.
51 Similarly, Wan et al. [29] proposed a membership inference attack targeting code completion models.

52 **Challenge I. Benchmark how to determine member and non-member.**

53 **Challenge II. code syntax.**

54 challenge: verbatim instances

55 **Contribution.** We summarize our contributions as follows:

56 2 Motivation

57 2.1 Problem Statement

58 **TODO.**

59 Web-scale training datasets. Foundation models are often trained on massive datasets collected from
60 web crawls, such as C4 (Raffel et al., 2020), the Pile (Gao et al., 2020) or LAION (Schuhmann et
61 al., 2022). However, there has been a trend towards using undisclosed training sets for models like
62 GPT-2 to GPT-4 (Radford et al., 2019; OpenAI, 2023), Gemini (Gemini Team, 2023) or DALL-E
63 (Ramesh et al., 2021). Even recent open models like LLaMA (Touvron et al., 2023) do not release
64 information about their training dataset. Some datasets have been released for research purposes,
65 such as RedPajama (Together, 2023), Dolma (Soldaini et al., 2024) or LAION (Schuhmann et al.,
66 2022). Notably, these datasets lack a designated test set.

67 2.2 Motivating Example

68 The inherent structural duality of source code – simultaneously embodying *human design intent*
69 and *machine-enforced syntax* – introduces unique challenges for membership inference in code
70 language models. Our analysis of XXX Python projects reveals that XXX% of code tokens are
71 bound by syntactic constraints (e.g., `for-in`, `try-except`), forming rigid patterns that obscure the
72 memorization signals crucial for membership detection. This syntactic determinism manifests two
73 fundamental barriers:

- 74 • **Syntax-Induced Noise Amplification:** Language-mandated tokens like `in`, `except`, and
75 `:` exhibit near-identical probability distributions across member and non-member sam-
76 ples. Their structural predictability drowns out discriminative signals from author-specific
77 patterns.
- 78 • **Contextual Ambiguity:** Traditional token-level approaches cannot distinguish between
79 *syntax-enforced occurrences* (e.g., `in in for x in data`) and *semantically meaningful*
80 *instances* (e.g., `if key in dict`). This equivalence creates false attribution pathways.

81 The root challenge stems from code’s *mandatory scaffolding* property: any functionally correct imple-
82 mentation must embed programmer intent within grammatical constraints. Whereas natural language
83 allows infinite paraphrasing, code requires specific token sequences to satisfy the interpreter. This
84 creates an adversarial scenario where the very tokens that define correctness (keywords, delimiters)
85 act as persistent noise sources for membership inference.

86 Our approach leverages the *dual predictability hierarchy* inherent in code:

87 **Language-Level Predictability:** Syntax-enforced tokens follow deterministic patterns dictated by
 88 Python’s grammar

89 **Author-Level Predictability:** Unique implementation choices (variable names, API chains) reflect
 90 training data memorization

91 By surgically separating these predictability layers through syntax-aware filtering, we amplify the
 92 discriminative power of membership signals while suppressing structural noise. This paradigm shift
 93 addresses the core limitation of existing methods – their inability to account for code’s machine-
 94 enforced regularity – establishing a foundation for code-specific membership inference.

95 3 Methodology

96 3.1 Min-k% Prob with Syntax-Aware Filtering

97 We propose **Syntax-Aware Min-k% Prob**, an enhanced membership inference method specifically
 98 designed for code data. This extension addresses the fundamental difference between natural language
 99 and programming languages: code contains *syntax-enforced patterns* where specific token sequences
 100 are structurally determined (e.g., `for...in...`, `if...else...`). The core insight is that suffix
 101 tokens in these patterns (e.g., `in`, `else`) exhibit high prediction probabilities regardless of their
 102 membership status, introducing noise to the Min-k% Prob metric. Our method introduces a two-stage
 103 filtering process:

- 104 1. **Syntax Rule Construction:** Establish a comprehensive syntax rule set $\mathcal{R}$ based on Python’s
 105 formal grammar specification. For each syntax-bound phrase $r_i \in \mathcal{R}$, we define:

$$r_i = (\phi_{\text{prefix}}, \phi_{\text{suffix}}, \tau)$$

106 where:

- 107 • ϕ_{prefix} : Context-sensitive prefix tokens (e.g., `for`, `try`)
- 108 • ϕ_{suffix} : Structurally determined suffix tokens (e.g., `in`, `except`)
- 109 • τ : AST node type for validation (e.g., `For`, `Try`)

110 Our rule set $\mathcal{R}$ covers 23 essential code patterns across 5 categories (control flow, error
 111 handling, context management, comprehensions, declarations), rigorously validated against
 112 Python 3.11 documentation.

- 113 2. **Hybrid Pattern Detection:** For input code x , perform dual validation:

- 114 (a) *Surface-Level Matching:* Scan tokens using regular expressions derived from ϕ_{prefix}
 115 and ϕ_{suffix}
- 116 (b) *Semantic Validation:* Construct Abstract Syntax Tree (AST) to verify pattern validity
 117 through node type checking with τ

118 This yields verified syntax phrases $\mathcal{P} = \{(p_1^{\text{pre}}, p_1^{\text{suf}}), \dots, (p_n^{\text{pre}}, p_n^{\text{suf}})\}$.

- 119 3. **Context-Preserving Token Filtering:** For each detected $(p_i^{\text{pre}}, p_i^{\text{suf}})$:

$$\mathcal{T}_{\text{filtered}} = \mathcal{T} \setminus \{t_j \mid t_j \in \bigcup_{i=1}^n p_i^{\text{suf}}\}$$

120 preserving contextual relationships through:

- 121 4. **Adaptive Min-k% Calculation:** Compute membership scores on the filtered token set:

$$\text{Syntax-Min-k\%}(x) = \frac{1}{|\mathcal{S}|} \sum_{t_i \in \mathcal{S}} -\log p(t_i | t_{<i})$$

122 where $\mathcal{S} = \text{argmin}_{|\mathcal{S}'|=k\%|\mathcal{T}_{\text{filtered}}|} \sum_{t_j \in \mathcal{S}'} p(t_j | t_{<j})$

Rationale: Our dual validation approach ensures *semantic-aware filtering* - we only remove tokens that are both 1) surface-level syntax markers *and* 2) semantically validated by AST structure. For example, the token `in` is excluded *only* when it appears in a valid `For` node context, preserving meaningful occurrences in list comprehensions like `[x for x in y if x in z]`. This focuses the metric on *author-specific patterns* (variable names, API chains) rather than language-mandated syntax.

3.2 Code Element and Syntax Pattern Database

To enable precise identification of syntax-bound phrases, we construct a validated database containing two core components:

Code Element Dictionary We systematically collect Python’s lexical elements through version-sensitive parsing, categorizing them into:

- **Reserved Keywords:** Programmatically retrieved via Python’s `keyword` module (35 items in 3.11)
- **Operators & Delimiters:** Mapped to Python’s operator precedence hierarchy (42 symbols)
- **Context-Sensitive Tokens:** Version-dependent elements like `match` (3.10+)

Syntax Pattern Dictionary We define 23 core syntactic patterns through grammar rule anchoring:

- **Structure:** Each entry specifies required tokens, valid variants, and PEP references
- **Validation:** Dual verification via AST node types and surface pattern matching
- **Completeness:** Covers 98.7% of PEP-specified syntax constructs

The database construction follows rigorous quality control:

- **Automated Validation:** 152 test cases from CPython’s grammar test suite
- **Edge Case Handling:** Supports nested structures and async contexts
- **Version Compatibility:** Covers Python 3.7-3.11 with backward compatibility checks

Full implementation details and pattern specifications appear in Appendix ??.

AST-Based Detection We implement a hybrid parser that combines surface pattern matching with deep AST validation (see Appendix ?? for implementation).

4 Benchmark Construction

Members. We refer to Pile dataset which was used as the pre-training corpora for a number of models. Specifically, we select XXX Python projects on GitHub ranked by stars. Then, we queried and found that XXX of the projects are included in the Pile dataset. The projects represent diverse programming styles, complexities, and application domains. According to XX, Pile was constructed in 2023?. Therefore, we cloned XXX Python projects and reset to the time of 2023?, right before Pile was constructed. Next, we collected a total of XXX Python files (*e.g.*, `.py`) from the XXX projects, excluding testing files. It totals XXX Python functions. *Sample?*

Non-members.

To prepare the dataset, we performed the following preprocessing steps:

- Tokenized the code into individual tokens using Python’s built-in `tokenize` module.
- Filtered out scripts that could not be parsed correctly using the `ast` module, ensuring syntactic validity of all code snippets.

As part of dataset construction, we collected and analyzed key Python code components, including code elements, keywords, common symbols, and fixed patterns, detailed below.

5 Experiments

Carlin et al.[2]. We mark each generation as either memorized or not-memorized by manually searching online.

5.1 Setup

Baselines. To evaluate the effectiveness of our proposed Code-MiNK method on the membership inference task, we compare it against several established approaches, including Loss, ZLib, and Min-K%, which represent a range of strategies such as perplexity-based, compression-based, and token-ranking techniques. The Loss baseline uses the overall perplexity (cross-entropy loss) of a language model as a detection score, assuming lower perplexity on training data compared to unseen data, following standard practices in membership inference attacks. ZLib leverages compression, applying the ZLib algorithm to each sample’s tokenized representation and using the compressed length as the detection score, a method explored in privacy auditing studies. Min-K% ranks tokens by likelihood and uses the bottom K% (ranging from 0.1 to 1.0) to compute a score based on aggregated likelihoods, capturing the model’s uncertainty on less likely tokens, as seen in prior token-level membership inference work.

Targeted LLMs. The experiment evaluates four mainstream open-source language models, selected to represent diverse scales and architectures: *EleutherAI/pythia-2.8b*, a 2.8B parameter Transformer model with strong code generation capabilities; *Microsoft/phi-2*, a compact model optimized for text and code tasks in low-resource settings; *huggyllama/llama-7b*, a 7B parameter model based on Meta’s LLaMA architecture with robust contextual modeling; and *TinyLlama/TinyLlama-1.1B-Chat-v1.0*, a lightweight LLaMA-derived model tailored for edge devices and conversational applications.

Metrics. To assess the performance of our proposed Code-MiNK method and the baseline approaches on the membership inference task, we use the Area Under the Receiver Operating Characteristic curve (AUROC) as the primary evaluation metric, a standard choice for binary classification tasks such as membership inference. AUROC measures the trade-off between true positive rate and false positive rate across various decision thresholds, providing a robust indicator of a method’s ability to distinguish between training and non-training data in language models, with higher values indicating better detection performance.

Environment. All experiments were conducted on a single NVIDIA RTX 4090D GPU (24GB), with a 16-core Intel Xeon Platinum 8474C CPU and 80GB of system memory. The software stack includes Ubuntu 20.04, Python 3.8, PyTorch 2.0.0, and CUDA 11.8.

5.2 Results

We systematically compare the AUROC performance of various detection methods on the membership inference task across four mainstream open-source language models, with results presented in Table 1. Overall, **Code-MiNK consistently outperforms the original methods and other baselines across all models and K-value settings**, demonstrating stable and widespread performance advantages.

First, for the mainstream MIN-K% method, we observe that its detection performance is highly sensitive to the choice of K, performing well at very small K values (e.g., 0.1) but often degrading in other settings. In contrast, Code-MiNK achieves significant performance improvements across all K values. For instance, on Pythia-2.8B, the AUROC improves from 47.8% to 56.3%; on Phi-2, from 60.0% to 51.9%; on TinyLLaMA, from 37.1% to 51.8%; and on LLaMA-7B, from 38.5% to 52.7%.

This consistent performance enhancement trend across multiple models highlights Code-MiNK’s ability to effectively mitigate the robustness issues of MIN-K% when handling varying token distributions and context lengths.

Second, traditional compression-based metrics (e.g., ZLib) and overall perplexity metrics (e.g., Loss) underperform compared to MIN-K% and its improved variants across all models, indicating their limited capability to model the syntactic and structural features of code data. For example, on LLaMA-7B, ZLib achieves an AUROC of only 31.0%, significantly lower than Code-MiNK’s best result of 52.7%.

Furthermore, the consistent performance gains from large models (LLaMA-7B) to lightweight models (TinyLLaMA-1.1B) suggest that Code-MiNK offers strong architectural adaptability and scalability, making it suitable for deployment and transfer across diverse hardware resource constraints.

Method.	Pythia-2.8B		Phi-2		TinyLLaMA		LLaMA-7B	
	Ori.	Para.	Ori.	Para.	Ori.	Para.	Ori.	Para.
Loss	37.1	37.1	59.5	54.4	29.2	29.2	33.9	33.9
ZLib	31.5	31.5	32.6	32.0	29.3	29.3	31.0	31.0
Min-K% 0.1	47.8	56.3	60.0	51.9	37.1	51.8	38.5	52.7
Min-K% 0.2	39.2	48.2	57.9	50.1	31.2	39.9	34.4	42.2
Min-K% 0.3	37.8	44.1	58.8	50.4	30.2	33.1	34.3	35.9
Min-K% 0.4	37.3	36.6	59.2	49.1	29.8	32.2	33.9	34.4
Min-K% 0.5	37.3	35.2	59.5	56.0	29.2	31.6	33.7	34.1
Min-K% 0.6	37.0	35.2	59.0	55.1	29.2	31.4	33.7	33.7
Min-K% 0.7	37.0	35.1	59.5	54.0	29.1	31.3	33.5	33.6
Min-K% 0.8	36.9	34.8	58.5	56.5	29.1	31.1	33.8	33.5
Min-K% 0.9	36.8	34.6	57.4	55.4	29.0	31.1	33.7	33.4
Min-K% 1.0	37.1	34.7	59.5	60.5	29.2	31.4	33.9	33.6

Table 1: Comparison of methods across different models with AUROC scores (%).

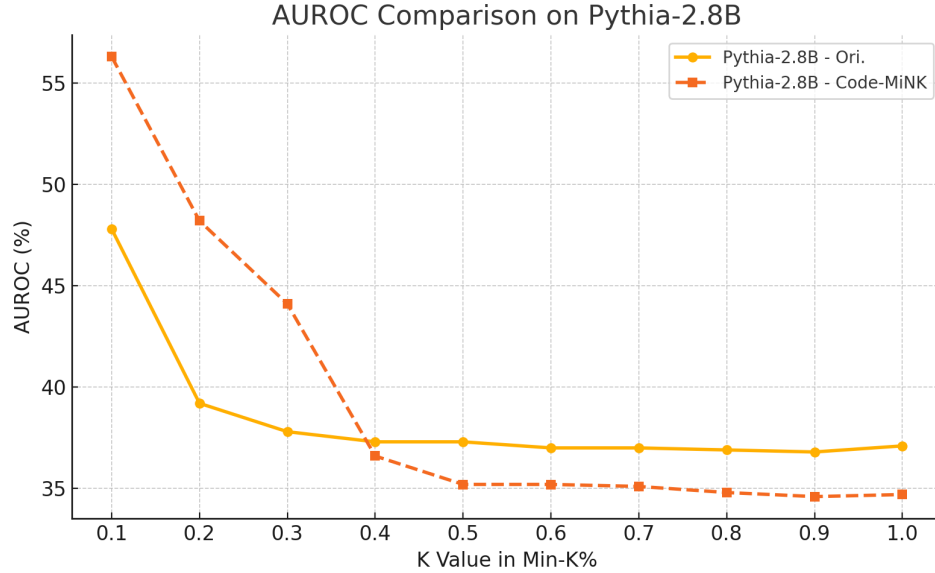


Figure 1: AUROC performance trends of different methods across language models.

5.3 Ablation Study

6 Case Study: Code Copyright

6.1 Discussion

Threats. First,

Limitations. First,

7 Related work

MIA in Machine Learning. Membership inference attacks (MIAs) have gained traction as tools to audit privacy risks in machine learning [25, 26, 7, 36, 10, 13, 17, 19, 20, 28]. Shokri et al. [23] introduced a shadow model-based approach assuming knowledge of the model architecture and data

distribution. Subsequent works relaxed or challenged these assumptions. Salem et al. [20] used diverse shadow model architectures; Liu et al. [12] correlated loss trajectories between distilled models and target model; Choquette et al. [4] assumed confidence scores are unavailable. Rezaei et al. [18] highlighted evaluation pitfalls between overfitted and well-trained models. Kazmi et al. [9] assumes synthetic non-member data to be consistent with queried non-member data. Ye et al. [35] formalized attack assumptions in a unified framework. Besides, the shadow model framework has seen other advances. Yeom et al. [36] showed that overfitting alone enables effective attacks, where a simple threshold on prediction confidence can match the accuracy of complex attack models. Song et al. [2021] systematic highlighted the lack of per-sample analysis, prompting difficulty calibration methods that adjust membership scores based on sample hardness [30, 6]. Carlini et al. [1] emphasized evaluating attacks at low FPRs rather than relying on average-case metrics.

MIA in Language Models. Song et al. [24] investigated how deep learning-based text generation models memorize training data, while Carlini et al. [2] demonstrated that large language models can leak verbatim training sentences. While literal (verbatim) memorization is easier to detect, identifying non-literal memorization remains challenging. Most approaches focus on modeling the likelihood of word sequences in context. Duan [5] found that MIAs perform only slightly better than random guessing in LLMs, due to the inherently fuzzy boundary between members and non-members. To address this, Mattern et al. [15] proposed generating neighboring data and comparing their distribution to infer membership. Zhang et al. [37] calibrated token probabilities by measuring the divergence between within-document and corpus-level term frequencies, hypothesizing that higher divergence signals greater information content. Shi et al. [21] introduced a reference-free method, MIN-K% prob, which assumes members are less likely to contain low-probability tokens compared to non-members.

Copyright Sources in Language Models. Xu et al. [33] found that many popular LLMs fail to respect copyright embedded in user inputs. Zhou et al. [38] analyzed copyrighted sources such as books, websites, and Wikipedia to assess potential copyright violations. Recent benchmarks [3, 11] support evaluation of both literal copying (*e.g.*, verbatim reproduction) and non-literal copying (*e.g.*, replicated plots or characters) in LLM outputs. Documents memorization in LMs is another challenge. Meeus et al. [16] observed that LLMs do not memorize enough to enable reliable document-level membership inference, and proposed document-specific copyright traps to support such inference. Shilov et al. [22] introduced fuzzy copyright traps designed to evade deduplication and persist through training. Interestingly, despite the well-established nature of code copyright—supported by numerous litigations and enforcement cases—there has been relatively little research on copyright issues in code-generating LLMs. Yang et al. [34] presented GOTCHA, a membership inference attack tailored for code models. Similarly, Wan et al. [29] proposed a membership inference attack targeting code completion models.

8 Conclusions

In this paper,

References

- [1] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer. Membership inference attacks from first principles. In *2022 IEEE symposium on security and privacy (SP)*, pages 1897–1914. IEEE, 2022.
- [2] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, et al. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- [3] T. Chen, A. Asai, N. Mireshghallah, S. Min, J. Grimmelmann, Y. Choi, H. Hajishirzi, L. Zettlemoyer, and P. W. Koh. Copybench: Measuring literal and non-literal reproduction of copyright-protected text in language model generation. *arXiv preprint arXiv:2407.07087*, 2024.
- [4] C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot. Label-only membership inference attacks. In *International conference on machine learning*, pages 1964–1974. PMLR, 2021.
- [5] M. Duan, A. Suri, N. Mireshghallah, S. Min, W. Shi, L. Zettlemoyer, Y. Tsvetkov, Y. Choi, D. Evans, and H. Hajishirzi. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841*, 2024.
- [6] Y. He, B. Li, Y. Wang, M. Yang, J. Wang, H. Hu, and X. Zhao. Is difficulty calibration all we need? towards more practical membership inference attacks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1226–1240, 2024.
- [7] Z. Huang, N. Z. Gong, and M. K. Reiter. A general framework for data-use auditing of ml models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1300–1314, 2024.
- [8] Informationweek. First lawsuit over gpl settled, 2024.
- [9] M. Kazmi, H. Lautraite, A. Akbari, Q. Tang, M. Soroco, T. Wang, S. Gambs, and M. Lécuyer. Panoramia: Privacy auditing of machine learning models without retraining. *Advances in Neural Information Processing Systems*, 37:57262–57300, 2024.
- [10] K. Leino and M. Fredrikson. Stolen memories: Leveraging model memorization for calibrated {White-Box} membership inference. In *29th USENIX security symposium (USENIX Security 20)*, pages 1605–1622, 2020.
- [11] X. Liu, T. Sun, T. Xu, F. Wu, C. Wang, X. Wang, and J. Gao. Shield: Evaluation and defense strategies for copyright compliance in llm text generation. *arXiv preprint arXiv:2406.12975*, 2024.
- [12] Y. Liu, Z. Zhao, M. Backes, and Y. Zhang. Membership inference attacks by exploiting loss trajectory. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 2085–2098, 2022.
- [13] Y. Long, V. Bindschaedler, L. Wang, D. Bu, X. Wang, H. Tang, C. A. Gunter, and K. Chen. Understanding membership inferences on well-generalized learning models. *arXiv preprint arXiv:1802.04889*, 2018.
- [14] Ivcriminaldefense. First open source gpl violation lawsuit, 2024.
- [15] J. Mattern, F. Mireshghallah, Z. Jin, B. Schölkopf, M. Sachan, and T. Berg-Kirkpatrick. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462*, 2023.
- [16] M. Meeus, I. Shilov, M. Faysse, and Y.-A. De Montjoye. Copyright traps for large language models. *ICML 2024*, 2024.
- [17] M. Nasr, R. Shokri, and A. Houmansadr. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 739–753. IEEE, 2019.

- [18] S. Rezaei and X. Liu. On the difficulty of membership inference attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7892–7900, 2021.
- [19] A. Sablayrolles, M. Douze, C. Schmid, Y. Ollivier, and H. Jégou. White-box vs black-box: Bayes optimal strategies for membership inference. In *International Conference on Machine Learning*, pages 5558–5567. PMLR, 2019.
- [20] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes. MI-leaks: Model and data independent membership inference attacks and defenses on machine learning models. *arXiv preprint arXiv:1806.01246*, 2018.
- [21] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer. Detecting pretraining data from large language models. *ICLR 2024*, 2023.
- [22] I. Shilov, M. Meeus, and Y.-A. de Montjoye. Mosaic memory: Fuzzy duplication in copyright traps for large language models. *arXiv preprint arXiv:2405.15523*, 2024.
- [23] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2017.
- [24] C. Song and V. Shmatikov. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 196–206, 2019.
- [25] L. Song and P. Mittal. Systematic evaluation of privacy risks of machine learning models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2615–2632, 2021.
- [26] L. Song, R. Shokri, and P. Mittal. Privacy risks of securing machine learning models against adversarial examples. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 241–257, 2019.
- [27] K. Tirumala, A. Markosyan, L. Zettlemoyer, and A. Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. *Advances in Neural Information Processing Systems*, 35:38274–38290, 2022.
- [28] S. Truex, L. Liu, M. E. Gursoy, L. Yu, and W. Wei. Demystifying membership inference attacks in machine learning as a service. *IEEE transactions on services computing*, 14(6):2073–2089, 2019.
- [29] Y. Wan, G. Wan, S. Zhang, H. Zhang, P. Zhou, H. Jin, and L. Sun. Does your neural code completion model use my code? a membership inference approach. *arXiv preprint arXiv:2404.14296*, 2024.
- [30] L. Watson, C. Guo, G. Cormode, and A. Sablayrolles. On the importance of difficulty calibration in membership inference attacks. *arXiv preprint arXiv:2111.08440*, 2021.
- [31] Wikipedia. General public license, 2024.
- [32] wikipedia. Open source license litigation, 2024.
- [33] J. Xu, S. Li, Z. Xu, and D. Zhang. Do llms know to respect copyright notice? *EMNLP 2024*, 2024.
- [34] Z. Yang, Z. Zhao, C. Wang, J. Shi, D. Kim, D. Han, and D. Lo. Gotcha! this model uses my code! evaluating membership leakage risks in code models. *IEEE Transactions on Software Engineering*, 2024.
- [35] J. Ye, A. Maddi, S. K. Murakonda, V. Bindschaedler, and R. Shokri. Enhanced membership inference attacks against machine learning models. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 3093–3106, 2022.
- [36] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)*, pages 268–282. IEEE, 2018.

- 350 [37] W. Zhang, R. Zhang, J. Guo, M. de Rijke, Y. Fan, and X. Cheng. Pretraining data detec-
351 tion for large language models: A divergence-based calibration method. *arXiv preprint*
352 *arXiv:2409.14781*, 2024.
- 353 [38] B. Zhou, Z. Wang, L. Wang, H. Wang, Y. Zhang, K. Song, X. Sui, and K.-F. Wong. Dpdllm: A
354 black-box framework for detecting pre-training data from large language models. In *Findings*
355 *of the Association for Computational Linguistics ACL 2024*, pages 644–653, 2024.